Lab #1

Introduction to Embedded Systems

ASTRO 331 – Space Systems Engineering

Introduction

Embedded systems are at the heart of nearly every function in spacecraft—from controlling attitude and propulsion to managing power, communications, and payloads.

This lab will introduce you to embedded systems using a low-cost microcontroller and the Arduino development environment.

You'll perform four tasks that are each analogous to functions performed on spacecraft:

Task	Example Similar Spacecraft Function
(1) Real-Time Digital Output → Blink two LEDs at different rates	Concurrent control of multiple s/c subsystems
(2) Digital Input → Read pushbutton to control an LED	Check if a solar panel deployed properly
(3) Analog Input → Read value of photoresistor	Read data from a sun sensor
(4) Pulse Width Modulation (PWM) → Control brightness of LED using PWM	Modulate force produced by a thruster

Authorized Resources

- **This lab is individual effort.**
- You may use:
  - Any non-human resource
    - **Generative AI Level 4 Allowed**
  - Astro 331 course instructor

Objectives:

(1) Understand what embedded systems are, the function of their basic components, and why they are important for spacecraft.

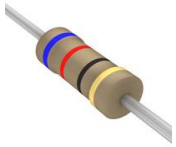
(2) Use a microcontroller development board and construct simple circuits on a breadboard.

(3) Write, upload, and debug basic embedded code to perform simple tasks.

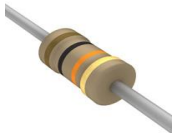
Materials (1) – Astro 331 Beginner Kit

- In this lab you'll use the Astro 331 Beginner Kit and your laptop.
- The beginner kit contains:
  - (1) Solderless breadboard
  - (2) Breadboard jumper wire kit
  - (3) Raspberry Pi Pico Microcontroller
  - (4) USB cable
  - (5) 3D Printed breadboard tool
  - (6) Basic electronic components:
    - (a) 6x LEDs
    - (b) 4x 10K Ohm Resistors (smaller resistors)
    - (c) 6x 62 Ohm Resistors (larger resistors)
    - (d) 2x photoresistors
    - (e) Push button
    - (f) Buzzer

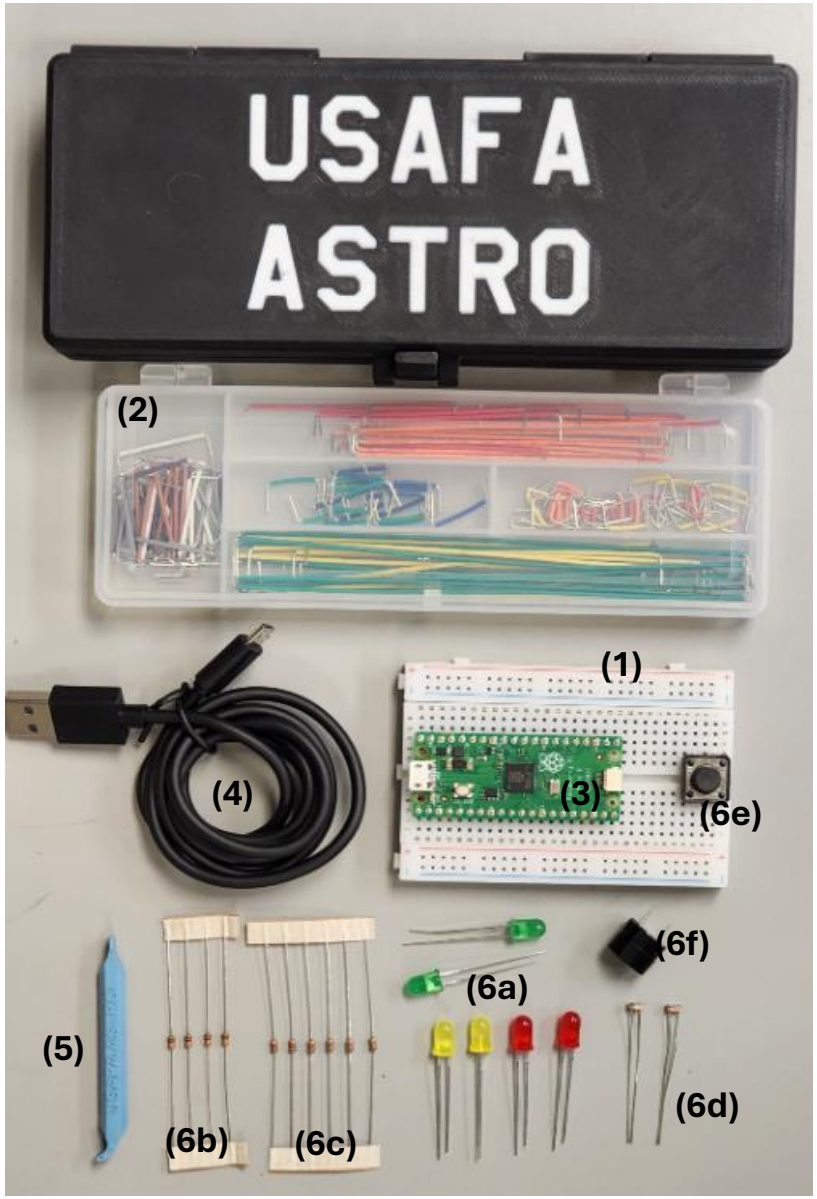
Photoresistor



62 Ohm Resistor (larger)



10K Ohm Resistor (smaller)



Materials (2) – Raspberry Pi Pico

The **Raspberry Pi Pico** is a low-cost microcontroller board based on the dual-core ARM Cortex-M0+ RP2040 chip, designed for embedded electronics and Internet of Things (IoT) applications.

Use the pinout diagram below throughout the lab to determine how to connect wires to the Pico.

WARNING

The Pico is a 3.3V microcontroller.
DO NOT apply >3.3V (i.e. 5V or 12V) to any pin on the device, except:
- 5V via the USB connecter (i.e. power from laptop or 5V charger)
- 5V to VBUS pin
- 1.8V-5.5V to VSYS pin

NOTE

The Raspberry Pi Pico can be programmed with either Micropython or C++. We'll be using the Arduino environment to program it in C++, but note that many examples online use Micropython.

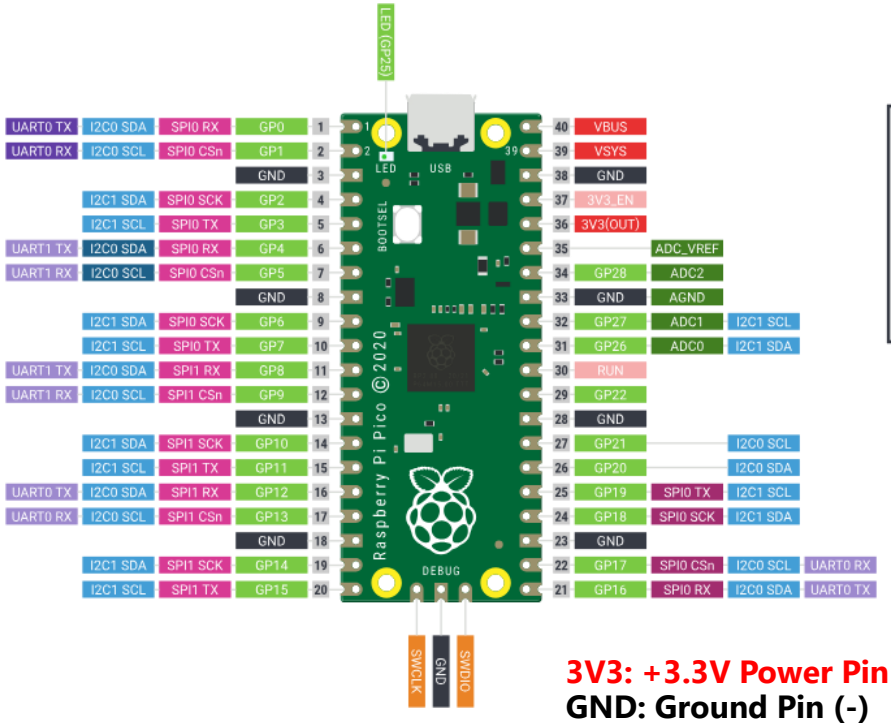
Raspberry Pi Pico Specifications:

- Dual-core Arm Cortex M0+ processor, flexible clock running up to 133 MHz
- 264KB of SRAM, and 2MB of on-board flash memory
- 26 × multi-function GPIO pins
- 2 × SPI, 2 × I2C, 2 × UART, 3 × 12-bit ADC, 16 × controllable PWM channels

Documentation: https://www.raspberrypi.com/documentation/microcontrollers/pico-series.html#pico-1-family

Datasheet: https://datasheets.raspberrypi.com/pico/pico-datasheet.pdf

Raspberry Pi Pico Pinout



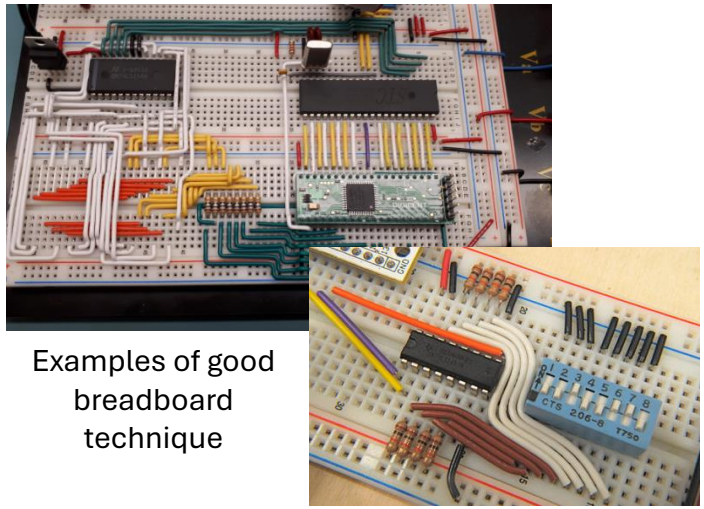
3V3: +3.3V Power Pin
GND: Ground Pin (-)
GPIO: General Purpose Input/Output
ADC: Analog to Digital Converter

Materials (3) – Solderless Breadboard

A breadboard is a reusable platform for prototyping electronic circuits without soldering.

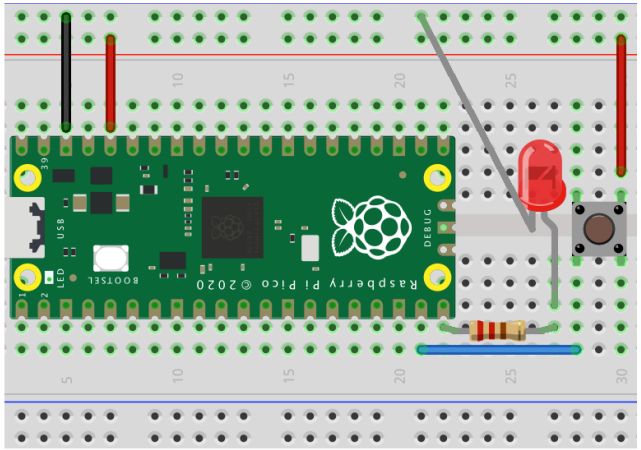
Interconnected rows and columns allow components to be connected via wires that are easily inserted and rearranged.

In addition to the breadboard, the beginner kit contains a jumper kit whose wires can be used to make connections on the breadboard.

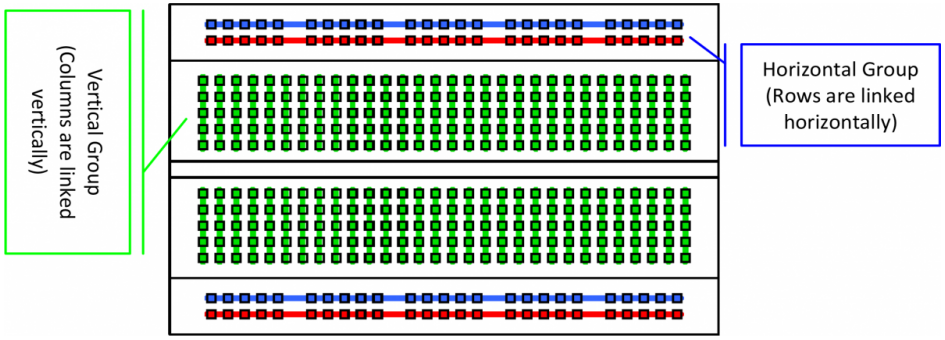


Examples of good breadboard technique

Jumper wire kit



Example wiring diagram for an LED and push button connected to a Pico



Electrical connections within breadboard

Further Reading: Adafruit Breadboads for Beginners

Task 0A – Self Study

- Spend at least 30 minutes using generative AI and other resources to teach yourself about embedded systems and electronics.
- Recommended Prompts:
  - What is an embedded system?
  - How are embedded systems used in Spacecraft?
  - What does 'real-time' mean in computing?
  - What is Arduino?
  - How does learning Arduino apply to Spacecraft Systems Engineering?
  - Explain the basic structure of an Arduino 'sketch'
  - Teach me about basic Arduino syntax
  - In Arduino, what does the millis() function do?
  - What is a Raspberry Pi Pico?
- As you explore these topics, **try entering the same prompt on at least two AI tools and/or a non-AI internet search.** Compare the results and note differences/inconsistencies.
- During this task and throughout the rest of the lab, consider pros and cons of using generative AI, and what tools work best for your learning.

Popular Generative AI Tools:
- ChatGPT
- Google Gemini
- Microsoft CoPilot
- Claude

WARNING

While generative AI can be a powerful learning aid, relying on it **without critical thinking** can lead to **misconceptions, oversimplified explanations, or factual errors**. AI-generated responses may sound confident even when incorrect, lack context, or miss nuances important to deep understanding. You should always:
- **Verify information with trusted sources**
- **Ask follow-up questions**
- Use AI as a **supplement—not a substitute—for rigorous study and active engagement**.

WARNING

Using generative AI for coding does not absolve you of the responsibility to understand what every line of code does and that the program functions as intended. Generated code almost always contains errors and/or is suboptimal. If you don't deeply understand code, don't use it!

Task Outcomes:
(1) Learn fundamentals of embedded systems and topics relevant to this course (Arduino, Raspberry Pi Pico, etc.)
(2) Gain exposure to generative AI, with a focus on understanding pros, cons, and potential pitfalls of using AI.

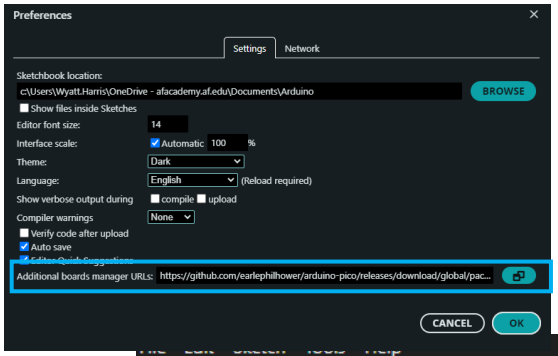
Concepts from this self study will be tested on the Lab 1 in-class quiz (closed book)

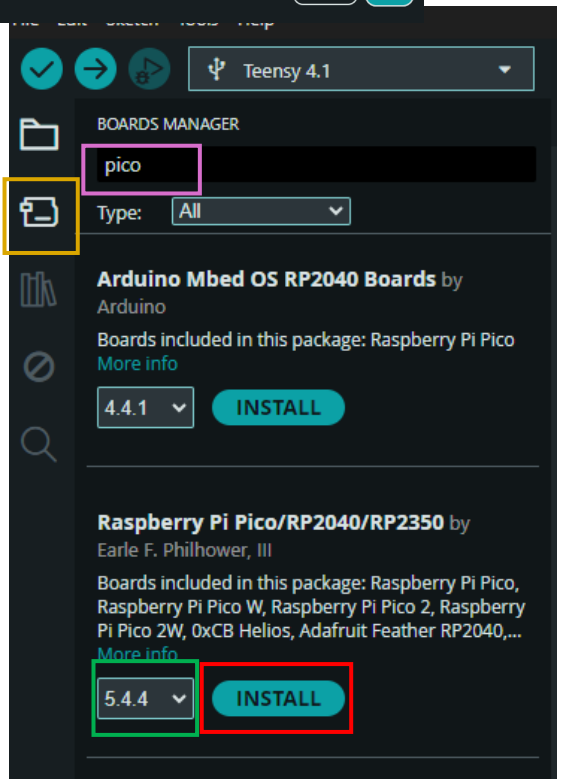
Task 0B – Install Arduino IDE

In Task 0B/0C, you will install the Arduino IDE, Raspberry Pi Pico board definition, and test uploading code to your Pico.

1. Install the Arduino Integrated Development Environment (IDE). [Version 2.3.7 as of writing]
   - 1.1 Download and run installer from here: https://www.arduino.cc/en/software/#ide

2. Open the Arduino IDE
3. Open **File → Preferences**
4. In **Additional Boards Manager URLs**, add:
   https://github.com/earlephilhower/arduino-pico/releases/download/global/package_rp2040_index.json
5. Install board definition for the Raspberry Pi Pico
   - 3.1 Select the Boards Manager icon
   - 3.2 Search for "pico"
   - 3.3 Select Version 5.4.4 (or latest version)
   - 3.4 Click Install (accept any pop-ups that appear)
6. Restart the Arduino IDE





Task 0C – Test Uploading Code to Your Pico

1. Connect your Raspberry Pi Pico.

   1.1 **New Raspberry Pi Pico's are loaded with Micropython by default. To use a new Pico with Arduino, hold down the 'BOOTSEL' button (see right) on the board the first time you plug it in to your computer with the provided USB cable.**

2. After plugging in the Pico, it should initially appear as a UF2_board in Arduino.

   2.1 Click the port drop down

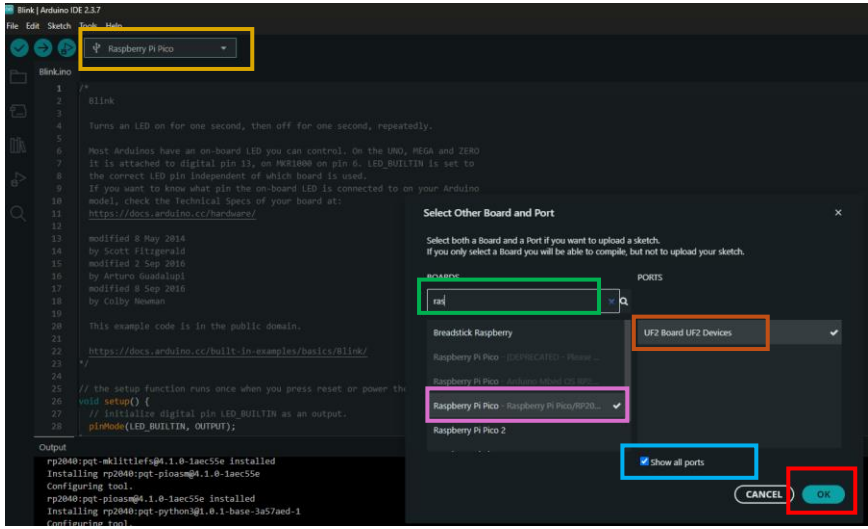
   2.2 Search for "raspberry pi pico"

   2.3 Select "Raspberry Pi Pico – Raspberry Pi Pico/RP2040"

   2.3 Select "Show all ports"

   2.4 Select "UF2 Board UF2 Devices" (you may have to select "show all ports" first

   2.5 Click OK



3. Write the following code between the curly brackets of the void loop() function:

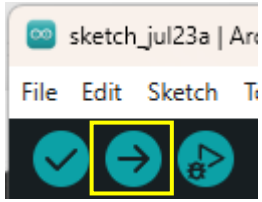
```
Serial.println("Hello World!");
delay(1000);
```

4. Click the upload button (arrow button on top left of IDE) and wait until "Done Uploading" is displayed.

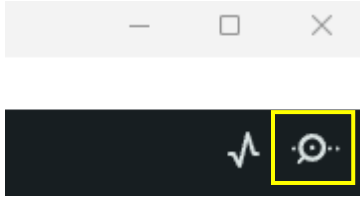
5. Open the serial monitor (top right of IDE)

   5.1 You should see "Hello World!" displayed once a second. This text is generated by the Pico and sent over a Serial connection to your computer.

6. The Pico should now show up as a COM port, and you will no longer need to hold the BOOLSEL button when you plug it in.



Upload button



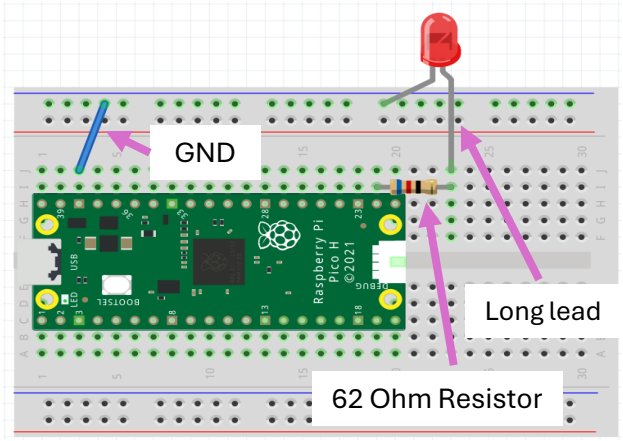
Serial Monitor

Task 1 – Real-Time Digital Output

In this task, you'll demonstrate real-time digital output by asynchronously blinking two LEDs using non-blocking code.

→ This is analogous to how spacecraft computers control multiple subsystems simultaneously.

1. Connect an LED to the Pico (see diagram at right)
   1.1 Connect one lead of a 62 Ohm resistor (larger resistor in your kit) to a GPIO pin of your choosing (note the pin number for your code).
   1.2 Connect the LED's long leg to the other end of the resistor, and the short leg to Ground.
2. Write code to change the state (on/off) of the LED every 500 milliseconds. Do not use the delay() function. Use the millis() function to create a timing statement that compares the current time to the previous time the LED's state was changed.
3. Add a second LED connected to a different GPIO pin.
4. Add a second timing statement like your first to blink the second LED simultaneously to the first at a different rate of your choosing.



Example Single LED circuit

WARNING

LEDs (light emitting diodes) illuminate when current flows through them. If connected directly to a voltage source, they will burn out immediately—**they must always be connected in series with a resistor, which will limit the current.**

CAUTION

LEDs are directional—the longer (or bent) leg must be connected to power, and the shorter leg to ground. Connecting the LED backwards won't damage anything, but the LED will not illuminate.

CAUTION

Using delay() in Arduino code can be problematic, especially for real-time or multitasking applications, because it blocks the processor from doing anything else during the delay period.

Task Outcome:

(1) Two LED's blinking simultaneously at different rates using non-blocking code (i.e Arduino delay() function is *not* used).

Task 2 – Digital Input

In this task, you'll use digital input from a pushbutton to control the state on an LED.

→ This is analogous to how a spacecraft checks if a physical event has happened, e.g. if a docking mechanism is attached or if a solar panel has deployed.

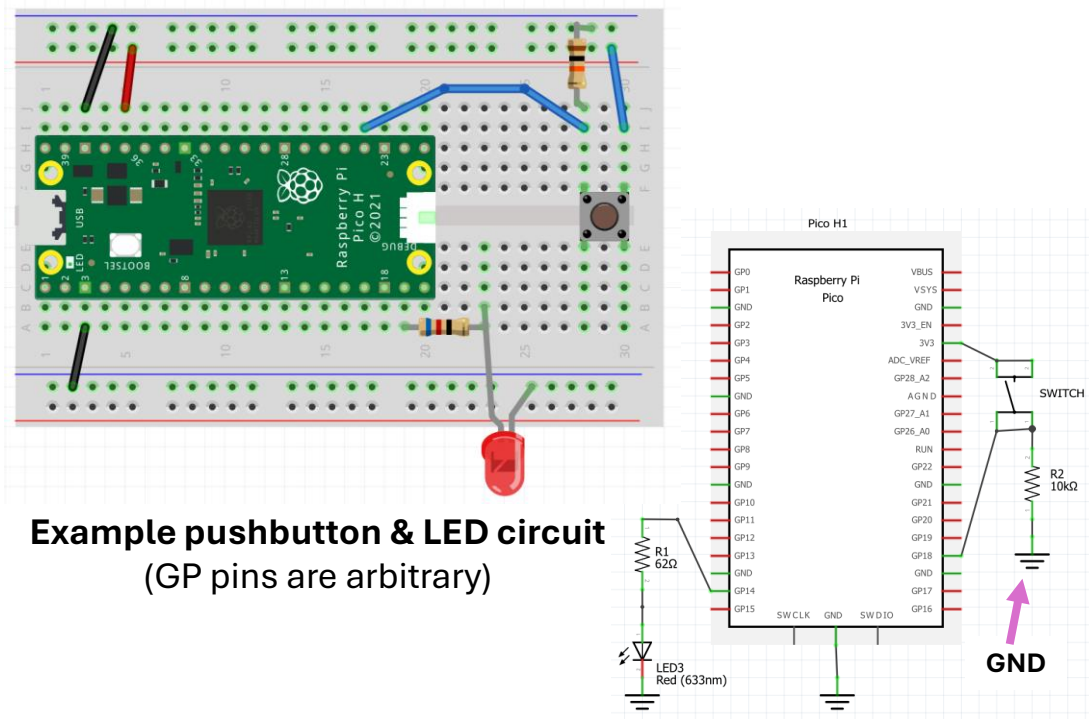
1. Keep LEDs from previous task connected and functional.
2. Connect a pushbutton and third LED to the Pico (reference diagram at right)
   1.1 Connect the third LED to a new GPIO as you did in the previous task.
   1.2 Connect the pushbutton and a pull-down resistor connected to ground as shown. The terminal of the button with the resistor should connect to a GPIO of your choosing. The other terminal connects to 3.3V power from the Pico.
3. Add code to your existing program to turn the LED on when the button is pushed and keep it off otherwise.
   3.1 Put this code in a separate timing statement as you did for each LED in the previous task and have it run at 100 Hz (i.e. the program reads the pushbutton state and updates the third LED every 10 msec).

Suggested GenAI Queries:

(1) Why is there a resistor connected between the pushbutton and ground?

(2) What is switch bounce and how do you fix it?

→Understand how we are dealing with switch bounce in this task.



Example pushbutton & LED circuit
(GP pins are arbitrary)

Task Outcomes:

(1) Previous task still functional.
(2) Pushbutton that controls state of third LED, updated 100 times a second.

Task 3 – Analog Input

In this task, you'll add a photoresistor to your circuit to sense the amount of light present.

→ **This is analogous to how spacecraft sun sensors work.**

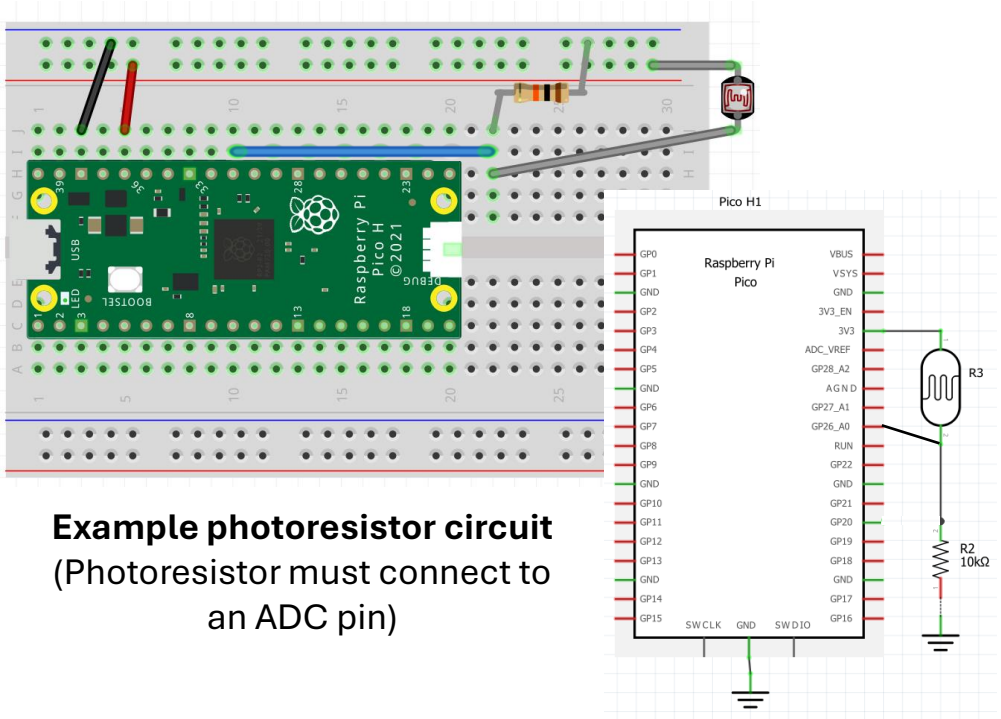
1. Keep components from previous tasks connected and functional.
2. Add a photoresistor and 10Kohm resistor (reference diagram at right). The GPIO pin that the photoresistor connects to is arbitrary but must be correctly declared in code.
3. While other tasks are still running, use analogRead() function to poll the photoresistor. Print the reading over Serial and view the reading .

   3.1. **As with previous tasks, put this code in a separate timing statement running at 10 Hz (i.e. the program reads the sensor and prints its value every 100 msec).**

   3.2. You can view the sensor reading using the Serial monitor, or alternatively, you can see it graphically using Tools→Serial Plotter.

Note

A **photoresistor**, also known as a **light-dependent resistor (LDR)**, is an electronic component whose **resistance changes based on the amount of light** that hits it. *It outputs a lower resistance the more light it senses.*



Example photoresistor circuit
(Photoresistor must connect to an ADC pin)

Task Outcome:
(1) Previous tasks still functional.
(2) Photoresistor connected with sensor reading printed to serial terminal 10 times a second.

Task 4 – Pulse Width Modulation (PWM)

In this task, you'll use Pulse Width Modulation (PWM) to vary the brightness of an LED based on the sensed amount of light from the photoresistor added in the last task.

→ PWM is used on spacecraft to control actuators such as thrusters or reaction wheels.

1. Keep components from previous tasks connected and functional.
2. Add a fourth LED/resistor to your circuit.
3. While other tasks are still running, control the brightness of this LED using a PWM signal whose duty cycle is determined by the sensor reading from the photoresistor.
   - 3.1. If the photoresistor reports a high value (low light), turn the LED on brighter, and vice versa (so if you place your finger over the resistor, the light should turn on).
   - 3.2. Perform a two-point calibration so that the LED is fully off when resistor is uncovered and LED is fully on when resistor is covered.
   - 3.3. **Place this code inside the same timing statement as the previous task (i.e. the LED's brightness is updated every time the photoresistor is read.)**

What is PWM?

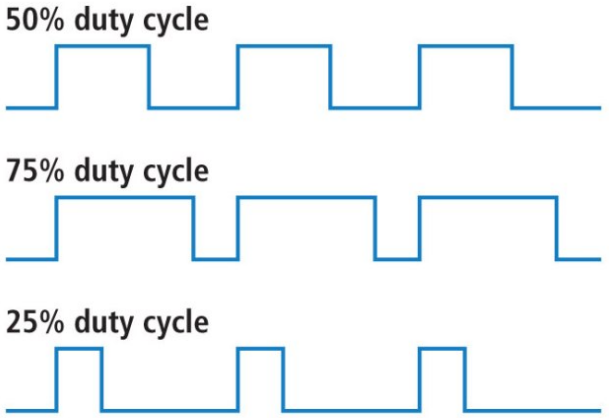
PWM (Pulse Width Modulation) is a technique used to simulate analog voltage using a digital output by rapidly switching it on and off at a fixed frequency, with a variable duty cycle.

How It Works:

Digital pins can only output HIGH (e.g., 5V or 3.3V) or LOW (0V).

PWM fakes an "average voltage" by turning the pin on and off very quickly.

The duty cycle determines how much time the signal is HIGH in each cycle.



50%, 75%, and 25% Duty Cycle Examples

Reference

Task Outcome:

(1) Previous tasks still functional.
(2) LED whose brightness is inversely proportional to amount of light sensed by photoresistor.

Turn-in Requirements

There are two parts to the turn-in for this lab:

(1) Send an email to your instructor.

- Subject of the the email: "LASTNAME Lab 1"
- Attach your Arduino .ino file that contains code for Tasks 1-4
  - Title your file "LASTNAME_Lab1.ino"
- Attach a short video (~15" is fine) demonstrating your code working:
  - Two LEDs blink at different rates
  - Press the push button to turn on a third LED
  - Put your finger over the photoresistor to show the fourth LED turn on
  - Title your video "LASTNAME_Lab1.---"
- Include a documentation statement in the body of the email
  - Specify which GenAI tools you used and how you used them (**at least 2 required**)
  - **Provide a link to your GenAI conversation(s)**

(2) An in-class closed-book quiz covering the content of this lab:

- Topics learned in Task 0A and throughout Tasks 1-4
- Basic functionality of code for Tasks 1-4 (you don't need to memorize Arduino syntax, but must understand how your code works)